

reference 学习重点总结

本文结合以下两份资料整理，并且尽量结合课件中的代码来解释：

- [05-synchronization.pdf](#)
- [lamport-actions.pdf](#)

这份笔记的目标不是逐页翻译，而是帮助你真正看懂：

- 代码在做什么
 - 为什么要这样写
 - 它解决了什么并发问题
-

1. 总体框架

这两份资料分别对应两个层面：

- **Synchronization**：教你“并发程序怎么写才不出事”
- **TLA**：教你“怎么形式化描述并证明并发程序是对的”

你可以把它们理解成：

- 前者偏实现
- 后者偏建模和验证

最好的学习顺序是：

1. 先看同步代码，理解 race condition、semaphore、monitor
 2. 再看 TLA，理解这些代码背后的 safety、liveness、fairness
-

2. Synchronization：结合代码理解重点

2.1 Race Condition：为什么并发代码会错

课件先给了一个非常典型的例子：两个线程同时修改同一个变量 `amount`。

高层代码看起来像这样：

```
// Thread 1
amount -= 10000;

// Thread 2
amount *= 0.5;
```

如果你只按“顺序程序”的思维去看，会觉得：

- 要么先减再乘

- 要么先乘再减
- 反正都应该是合理结果

但机器真正执行时，其实会拆成更细的步骤，例如：

```
// T1
r1 = load(amount);
r1 = r1 - 10000;
store(amount, r1);

// T2
r2 = load(amount);
r2 = 0.5 * r2;
store(amount, r2);
```

代码含义：

- `load(amount)`：把共享内存中的 `amount` 读到寄存器
- `store(amount, r1)`：再把寄存器结果写回共享内存

问题就在于：

- 读和写不是一个不可分割的动作
- 两个线程可以在中间穿插执行

错误交错可能导致：

- 两个线程都读到旧值 `100000`
- T1 算出 `90000`
- T2 算出 `50000`
- 最后谁后写回，谁覆盖前面的结果

这就是 `lost update`。

你要记住：

- `race condition` 不是“代码语句写错”
- 而是“多线程交错顺序导致共享状态被错误更新”

2.2 Critical Section：临界区的目标

课件把共享数据访问抽象成：

```
CSEnter();
CriticalSection();
CSExit();
```

代码含义：

- `CSEnter()`：进入临界区前的同步操作
- `CriticalSection()`：真正访问共享数据的代码
- `CSExit()`：离开临界区时释放同步资源

临界区设计的三个目标：

- **Safety**：同一时刻不能有多个线程同时不该进入却进入了
- **Liveness**：线程不会永远进不去
- **Fairness**：不要总让同一个线程占便宜

课件里最重要的一句话可以理解成：

- 临界区代码必须对所有可能的 interleaving 都安全

2.3 Too Much Milk：纯软件互斥为什么难

课件用买牛奶问题说明：即使只有两个“线程”，不用硬件原语也很难写对。

最原始写法：

```
if (fridge_empty())  
    buy_milk();
```

代码含义：

- 如果冰箱空了就去买牛奶

问题：

- 两个人可能都看到“冰箱空”
- 然后都去买

这破坏了 safety。

后来课件尝试加标志位、双标志位、非对称方案、Peterson 算法，本质都在做同一件事：

- 想办法让“我正在准备进临界区”这个信息被别人看到

你不一定要背每一种写法，但要记住结论：

- 只靠普通变量实现互斥很复杂
- 即便在两线程场景中也容易写错
- 所以系统最终需要更强的原子操作或更高层同步抽象

2.4 Test-And-Set 与锁

课件给出的硬件原语是：

```
ATOMIC int TestAndSet(int *var) {  
    int oldVal = *var;  
    *var = 1;  
    return oldVal;  
}
```

代码含义：

- 返回原来的值
- 同时把这个位置设成 1
- 整个过程是原子的，不能被线程切开

如果原来的值是：

- 0：说明原来没人占用，现在我成功拿到
- 1：说明已经有人占用，我拿不到

基于它可以写出锁：

```
acquire(int *lock) {  
    while (test_and_set(lock))  
        ;  
}  
  
release(int *lock) {  
    *lock = 0;  
}
```

代码含义：

- **acquire**：一直尝试，直到 test-and-set 返回 0
- **release**：把锁重新置为 0

问题也很明显：

- **while (...)** ; 是忙等
- 没抢到锁的线程会一直空转

所以这类锁叫 **spinlock**。

适用场景：

- 临界区极短
- 等待时间很短

不适用场景：

- 等待时间不确定
- 线程很多

- CPU 宝贵

2.5 Semaphore：P 和 V 到底是什么意思

课件中 semaphore 的抽象接口是：

```
P();    // procure / try to get resource
V();    // vacate / release resource
```

一个简化语义是：

```
P() {
    while (n <= 0) ;
    n -= 1;
}

V() {
    n += 1;
}
```

但这只是直观语义，不是高效实现。真正语义应该理解成：

- **P()**:
 - 如果资源够，就把计数减 1
 - 如果资源不够，线程阻塞等待
- **V()**:
 - 把计数加 1
 - 如果有线程在等，就唤醒一个

最关键理解：

- semaphore 自己“记得”资源数量
- 它不仅是唤醒工具，还是计数工具

2.6 Binary Semaphore：像锁一样用

课件中二值信号量常见写法：

```
Semaphore S;
S.init(1);

S.P();
CriticalSection();
S.V();
```

代码含义：

- `init(1)`：一开始资源有 1 份
- 第一个线程 `P()` 后变成 0，进入临界区
- 其他线程再 `P()` 就会被挡住
- 当前线程离开时 `V()`，让别人再进

这就是互斥。

所以：

- binary semaphore 可以充当 mutex

2.7 Counting Semaphore：像资源计数器一样用

课件给了一个“一个线程等另一个线程发通知”的例子：

```
Semaphore packetarrived;
packetarrived.init(0);

// ReceivingThread
pkt = get_packet();
enqueue(packetq, pkt);
packetarrived.V();

// PrintingThread
packetarrived.P();
pkt = dequeue(packetq);
print(pkt);
```

代码含义：

- 初始值 0 表示一开始没有包可打印
- 接收线程收到包后：
 - 把包入队
 - 调用 `V()` 表示“现在多了一个可消费事件”
- 打印线程调用 `P()`：
 - 如果还没有包，就阻塞
 - 如果已经来了包，就继续打印

这说明 counting semaphore 不只是锁，它还能表达：

- 有多少资源
- 有多少事件已经发生

2.8 Producer-Consumer：一定要结合代码理解

第一版：完全没保护

课件代码：

```
// add item to buffer
void produce(int item) {
    buf[in] = item;
    in = (in + 1) % N;
}

// remove item
int consume() {
    int item = buf[out];
    out = (out + 1) % N;
    return item;
}
```

代码含义：

- **buf**: 环形缓冲区
- **in**: 下一个写入位置
- **out**: 下一个读出位置
- **% N**: 到末尾后绕回开头

问题有两个：

1. 多个 producer / consumer 会同时改 **in/out/buf**
2. 没有检查“缓冲区是否为空/是否已满”

所以会出现：

- consumer 读空数据
- producer 覆盖未消费的数据

第二版：先保护共享资源

课件代码：

```
void produce(int item) {
    mutex_in.P();
    buf[in] = item;
    in = (in + 1) % N;
    mutex_in.V();
}

int consume() {
    mutex_out.P();
    int item = buf[out];
    out = (out + 1) % N;
    mutex_out.V();
}
```

```
    return item;
}
```

代码含义：

- `mutex_in`：保护写端状态
- `mutex_out`：保护读端状态

它解决了什么：

- `in` 的更新不会互相踩
- `out` 的更新不会互相踩

它还没解决什么：

- 缓冲区可能满了还写
- 缓冲区可能空了还读

所以这还不够。

第三版：再处理库存条件

课件标准版本：

```
void produce(int item) {
    space.P();      // need space
    mutex_in.P();
    buf[in] = item;
    in = (in + 1) % N;
    mutex_in.V();
    item.V();       // new item!
}

int consume() {
    item.P();       // need item
    mutex_out.P();
    int item = buf[out];
    out = (out + 1) % N;
    mutex_out.V();
    space.V();      // more space!
    return item;
}
```

虽然课件这里变量名 `item` 同时表示“信号量”和“局部变量”有点绕，理解时你可以把信号量想成：

```
Semaphore items(0);
Semaphore space(N);
```


真正含义是：

- `space.P()`：生产前先确认有空位
- `items.P()`：消费前先确认有数据
- 生产完成后 `items.V()`：通知“多了一个可消费元素”
- 消费完成后 `space.V()`：通知“多了一个空位”

这段代码的精髓是分工：

- `mutex_in / mutex_out`：保护共享状态修改的原子性
- `space / items`：管理资源数量

这就是经典的 bounded buffer 解法。

2.9 Readers-Writers：课件代码在做什么

课件的 semaphore 版本大意如下：

```
void write() {
    rw_lock.P();
    /* perform write */
    rw_lock.V();
}

int read() {
    count_mutex.P();
    rcount++;
    if (rcount == 1)
        rw_lock.P();
    count_mutex.V();

    /* perform read */

    count_mutex.P();
    rcount--;
    if (rcount == 0)
        rw_lock.V();
    count_mutex.V();
}
```

变量含义：

- `rcount`：当前正在读的读者数
- `count_mutex`：保护 `rcount`
- `rw_lock`：控制写者独占，或者“读者组”整体占有

writer 逻辑

```
rw_lock.P();  
/* write */  
rw_lock.V();
```

含义：

- 写者必须独占 `rw_lock`
- 这样保证写的时候没有别的写者，也没有读者

reader 进入逻辑

```
count_mutex.P();  
rcount++;  
if (rcount == 1)  
    rw_lock.P();  
count_mutex.V();
```

含义：

- 先安全地把读者数加一
- 如果我是第一个读者，就去拿 `rw_lock`
- 一旦第一个读者拿到了 `rw_lock`，写者就进不来了
- 后面的读者只需要增加 `rcount`，不用重复拿 `rw_lock`

所以：

- “第一位读者”负责挡住写者

reader 退出逻辑

```
count_mutex.P();  
rcount--;  
if (rcount == 0)  
    rw_lock.V();  
count_mutex.V();
```

含义：

- 读者离开时把计数减一
- 如果我是最后一个读者，就释放 `rw_lock`
- 这样写者才有机会进入

所以：

- “最后一位读者”负责把写者放进来

这段代码的优点和问题

优点：

- 多个读者可并发
- 写者独占

问题：

- 如果读者不断到来，写者可能一直抢不到锁
- 即 `writer starvation`

这就是课件强调的：

- 正确不代表公平

2.10 Semaphore 为什么容易错：课件里的经典错误

课件给了几种很危险的错误模式，虽然扫描文本有点乱，但本质很好理解。

错误 1：同一个线程连续 `P(S)` 两次

类似：

```
P(S);  
CriticalSection();  
P(S);
```

含义：

- 第一次 `P(S)` 已经拿到了资源
- 第二次又去拿一次
- 如果没有人帮你 `V(S)`，自己就把自己卡死了

这会造成：

- 自己死锁
- 后续线程也可能全挂住

错误 2：多写了一个 `V(S)`

类似：

```
P(S);  
CriticalSection();  
V(S);  
V(S);
```

含义：

- 你本来只该释放一次
- 结果多释放一次

后果：

- semaphore 计数被抬高
- 之后可能有多个线程同时通过
- 互斥性被破坏

错误 3：中途 `return` 导致少写一个 `V(S)`

类似：

```
P(S);  
if (x) return;  
CriticalSection();  
V(S);
```

含义：

- 进入临界区前拿了锁
- 结果中间提前返回
- 锁没释放

后果：

- 后面的线程全被堵死

你可以看出 semaphore 的根本风险：

- 同步逻辑散在代码流程里
- 一旦控制流变化，`P/V` 很容易不配对

这就是为什么课件会说：

- semaphore 是低层原语
- 小错误会导致系统挂死

2.11 Monitor：为什么更适合写共享对象

课件把 monitor 抽象成：

```
Monitor monitor_name {  
    // shared variable declarations  
    procedure P1() {}  
    procedure P2() {}  
}
```

```
...  
}
```

代码含义：

- 共享数据被封装进 monitor
- 对数据的操作也必须通过 monitor 提供的过程进入
- 同一时刻最多一个线程在 monitor 内运行

所以 monitor 带来的核心好处是：

- 锁的获取和释放不容易忘
- 共享状态不会在 monitor 外面被乱改
- 代码结构更像“对象 + 规则”

2.12 Condition Variable：代码真正表达的是“等条件”

课件强调 condition variable 的标准用法：

```
while (!some_predicate())  
    CV.wait();
```

代码含义：

- `some_predicate()`：我真正想等的条件
- `wait()`：如果条件还没满足，就睡眠

注意 `wait()` 的语义：

- 调用时线程睡眠
- 同时自动释放 monitor lock
- 被唤醒后重新拿回锁再继续

为什么必须是 `while` 而不是 `if`？

因为：

- 被唤醒不代表条件现在还成立
- 可能是伪唤醒
- 也可能是别的线程先一步把条件又改坏了

这点你一定要背熟。

2.13 BurgerKing 例子：最适合入门理解 monitor

课件代码大意：

```
class BK:
    def __init__(self):
        self.lock = Lock()
        self.hungrykid = Condition(self.lock)
        self.nBurgers = 0

    def make_burger(self):
        with self.lock:
            self.nBurgers = self.nBurgers + 1
            self.hungrykid.notify()

    def kid_eat(self):
        with self.lock:
            while self.nBurgers == 0:
                self.hungrykid.wait()
            self.nBurgers = self.nBurgers - 1
```

这个例子非常经典。

`__init__`

```
self.lock = Lock()
self.hungrykid = Condition(self.lock)
self.nBurgers = 0
```

含义：

- `lock`：monitor 的互斥锁
- `hungrykid`：孩子等待“有汉堡”的条件变量
- `nBurgers`：共享状态，记录汉堡数量

`make_burger`

```
with self.lock:
    self.nBurgers = self.nBurgers + 1
    self.hungrykid.notify()
```

含义：

- 厨师拿到锁后修改共享状态
- 汉堡数量加一
- 然后通知等待的孩子：“也许现在能吃了”

为什么是“也许”？

- 因为 `notify()` 不等于“条件必然满足给你了”

- 只是提示有线程可以起来重新检查

kid_eat

```
with self.lock:
    while self.nBurgers == 0:
        self.hungrykid.wait()
    self.nBurgers = self.nBurgers - 1
```

含义：

- 孩子先进 monitor
- 如果没有汉堡，就一直等
- 一旦醒来，再检查一次 `nBurgers == 0`
- 有汉堡才真正减 1 并吃掉

你要从这里学到 monitor 最关键的套路：

- 共享状态变量负责表达条件
- condition variable 只负责等和通知

2.14 Monitor 版 Producer-Consumer：条件写得更清楚

课件代码：

```
Monitor Producer_Consumer {
    char buf[SIZE];
    int n = 0, tail = 0, head = 0;
    condition not_empty, not_full;

    produce(char ch) {
        while (n == SIZE)
            wait(not_full);
        buf[head] = ch;
        head = (head + 1) % SIZE;
        n++;
        notify(not_empty);
    }

    char consume() {
        while (n == 0)
            wait(not_empty);
        ch = buf[tail];
        tail = (tail + 1) % SIZE;
        n--;
        notify(not_full);
        return ch;
    }
}
```

```
}  
}
```

变量含义：

- **n**：当前缓冲区已有元素数
- **head**：下一个写入位置
- **tail**：下一个读出位置
- **not_empty**：给消费者等“有货”
- **not_full**：给生产者等“有空位”

produce

```
while (n == SIZE)  
    wait(not_full);
```

含义：

- 如果缓冲区满了，生产者不能继续写
- 必须等到消费者消费之后再继续

接着：

```
buf[head] = ch;  
head = (head + 1) % SIZE;  
n++;  
notify(not_empty);
```

含义：

- 把元素写进去
- 头指针前进
- 元素数加一
- 通知消费者：“现在可能不空了”

consume

```
while (n == 0)  
    wait(not_empty);
```

含义：

- 没有数据就不能消费

接着：


```
ch = buf[tail];
tail = (tail + 1) % SIZE;
n--;
notify(not_full);
```

含义：

- 取出元素
- 尾指针前进
- 元素数减一
- 通知生产者：“现在可能有空位了”

这版代码比 semaphore 版更容易读，因为：

- 你能直接看出线程在等什么条件
- 条件写在 `while (n == SIZE) / while (n == 0)` 里

2.15 Monitor 版 Readers-Writers：公平性写进条件里

课件代码大意：

```
Monitor ReadersNWriters {
    int waitingWriters = 0, waitingReaders = 0;
    int nReaders = 0, nWriters = 0;
    Condition canRead, canWrite;

    BeginWrite() {
        ++waitingWriters;
        while (nWriters > 0 || nReaders > 0)
            canWrite.wait();
        --waitingWriters;
        nWriters = 1;
    }

    EndWrite() {
        nWriters = 0;
        if (waitingWriters > 0)
            canWrite.signal();
        else if (waitingReaders > 0)
            canRead.broadcast();
    }

    BeginRead() {
        ++waitingReaders;
        while (nWriters > 0 || waitingWriters > 0)
            canRead.wait();
        --waitingReaders;
        ++nReaders;
    }
}
```

```
EndRead() {  
    --nReaders;  
    if (nReaders == 0 && waitingWriters > 0)  
        canWrite.signal();  
}
```

这段代码比 semaphore 版更适合学习，因为它把策略写明了。

writer 为什么这样等

```
while (nWriters > 0 || nReaders > 0)  
    canWrite.wait();
```

含义：

- 只要还有活跃写者或活跃读者，写者就不能进

writer 结束为什么优先唤醒 writer

```
if (waitingWriters > 0)  
    canWrite.signal();  
else if (waitingReaders > 0)  
    canRead.broadcast();
```

含义：

- 如果已经有写者在排队，就先放一个写者
- 否则再把读者们都叫起来

这体现了一种偏向写者、防止写者饿死的策略。

reader 为什么要检查 `waitingWriters > 0`

```
while (nWriters > 0 || waitingWriters > 0)  
    canRead.wait();
```

含义：

- 不仅当前不能有写者在写
- 连“有写者已经在排队”等待也不让新读者插队

这就是在改善公平性。

最后一个 reader 为什么唤醒 writer

```
if (nReaders == 0 && waitingWriters > 0)
    canWrite.signal();
```

含义：

- 只要还有读者活跃，写者都不能进
- 最后一个读者离开时，正好是写者进入的时机

2.16 Barrier：它不是互斥，而是“阶段同步”

课件的错误示例大意：

```
self.allCheckedIn = Condition(self.lock)

def checkin():
    with self.lock:
        nArrived += 1
        if nArrived < nThreads:
            while nArrived < nThreads:
                allCheckedIn.wait()
        else:
            allCheckedIn.broadcast()
```

这段代码想表达的是：

- 每个线程到达 barrier 后先把 `nArrived` 加一
- 没到齐就等
- 最后一个到的线程把其他线程全部唤醒

它体现的核心思想是对的：

- barrier 的条件不是“有资源”
- 而是“所有线程都到齐了”

你可以把 barrier 理解成：

- 大家先各做各的
- 做到某个阶段必须集合
- 全员到齐后才能进入下一轮

2.17 Hoare vs Mesa：为什么课件老强调 `while`

课件说大多数现实系统使用的是 `Mesa/Hansen` 语义。

Mesa 风格下：

- `signal()` 只是把等待线程放到 ready queue

- 不保证它立刻运行

所以当等待线程真正继续执行时：

- 条件可能已经又变了

因此：

```
while condition_not_true:
    cv.wait()
```

是必须的。

如果你写成：

```
if condition_not_true:
    cv.wait()
```

那线程醒来后就默认条件成立，风险很大。

这也是课件 commandments 里最重要的一条之一：

- `wait` 必须被条件包裹
- 而且要用 `while`，不要用 `if`

3. TLA：结合课件思想来理解

TLA 这一篇更偏理论，但你完全可以把它和前面的同步代码对应起来看。

3.1 TLA 想表达什么

Lamport 的核心思想是：

- 不要只看程序代码
- 要看“系统状态怎么变”

例如生产者消费者程序，TLA 不会先关心 C 或 Python 语法，而会关心：

- 当前缓冲区里有多少元素
- 哪些动作允许发生
- 哪些性质永远不能被破坏

3.2 State / Predicate / Action / Behavior

State

状态就是某一时刻所有变量的取值。

例如在 producer-consumer 中，一个状态可以由这些量描述：

- `n`
- `head`
- `tail`
- `buf`

Predicate

predicate 描述某个状态满足什么条件。

例如：

```
n >= 0
n <= SIZE
```

它们表达的是：

- 缓冲区元素数不会是负数
- 也不会超过容量

这类东西通常就对应程序里的不变量。

Action

action 描述一步状态转移。

例如把“生产一个元素”抽象成：

```
n' = n + 1
head' = (head + 1) % SIZE
```

它的意思就是：

- 执行这一步后，元素数增加
- 写指针前进

Behavior

behavior 是整个执行过程，也就是一串状态序列。

把它和代码联系起来看：

- 一次完整运行不是一条语句
- 而是一连串状态变化

3.3 Safety、Liveness、Fairness 怎么对应到前面的代码

Safety

坏事不会发生。

例如：

- producer 不会在满缓冲区继续写
- consumer 不会在空缓冲区继续读
- 两个 writer 不会同时写

Liveness

好事最终会发生。

例如：

- 只要有空间，生产者最终能生产
- 只要有数据，消费者最终能消费

Fairness

不要一直偏袒某一类线程。

例如：

- readers 不应该让 writer 永久饿死

所以 TLA 其实是在给你一种语言，把你在同步代码里口头说的这些正确性要求写成公式。

3.4 为什么 **Enabled** 很像“条件是否满足”

Lamport 定义的 **Enabled A** 可以通俗理解成：

- 动作 **A** 现在有没有资格执行

这和 monitor 代码很像。

比如在 producer-consumer 中：

- **produce** 是否 enabled，要看 $n < \text{SIZE}$
- **consume** 是否 enabled，要看 $n > 0$

所以你可以把：

- `while (n == SIZE) wait(not_full);`

理解成：

- “当前 produce 这个 action 还没 enabled，所以线程先等”
-

3.5 Weak Fairness 和 Strong Fairness

这部分最容易抽象，但也能和代码联系起来。

Weak Fairness

如果一个动作从某时刻开始一直都可执行，那么不能永远不执行。

类比：

- 缓冲区一直不满
- producer 一直都能生产
- 那么系统不应该永远不给 producer 机会

Strong Fairness

如果一个动作反复有机会执行，那么最终也该执行。

类比：

- 某个 writer 虽然不是一直都能写
- 但它反复等到“可写”的机会
- 如果系统永远都让它错过，那就不公平

所以：

- weak fairness 更适合“持续可执行”
- strong fairness 更适合“间歇性可执行但反复有机会”

3.6 为什么 TLA 强调 stuttering

这点和代码实现关系很大。

课件中的高层 monitor 代码看起来像一步：

```
with lock:
    self.nBurgers += 1
    self.hungrykid.notify()
```

但在底层实现里，它实际上可能分成很多细步骤：

- 拿锁
- 修改变量
- 放入 ready queue
- 释放锁

Lamport 说规格要允许 **stuttering**，本质上是为了让：

- 高层抽象的一步
- 能被底层多步实现

这样才能做 refinement。

你可以把它记成：

- 抽象规格看“关键状态变化”
 - 不必死盯实现中的每一个微步骤
-

4. 最值得背下来的代码理解模板

以后你看到任意同步代码，都可以按下面四个问题去拆：

1. 共享状态是什么

例如：

- `nBurgers`
- `n`
- `head/tail`
- `rcount`

2. 线程在等什么条件

例如：

- `nBurgers > 0`
- `n < SIZE`
- `n > 0`
- `nWriters == 0 && waitingWriters == 0`

3. 谁负责修改这个条件

例如：

- 厨师生产汉堡后让 `nBurgers` 增加
- 消费者消费后让 `n` 减少
- 最后一个 reader 离开时释放写者

4. 谁在条件变化后通知别人

例如：

- `notify(not_empty)`
- `notify(not_full)`
- `canWrite.signal()`
- `canRead.broadcast()`

如果你能把任意代码都按这四件事讲清楚，说明你已经真的理解同步代码了。

5. 最后总结

这两份资料合起来，真正想教你的其实是：

- 并发问题本质上是共享状态问题
- 同步代码的核心不是“调用了哪个 API”，而是“维护了哪些不变量”
- semaphore 能做很多事，但容易错
- monitor + condition variable 更容易把“线程在等什么条件”表达清楚
- TLA 则把这些正确性要求进一步抽象成 safety、liveness、fairness

如果你接下来还要继续学这部分，我最建议你做两件事：

1. 把 `producer-consumer` 和 `readers-writers` 两套代码手写并逐行解释
2. 把每段代码对应的 safety / liveness / fairness 自己写出来

这样你就不只是“看懂了”，而是已经具备自己分析同步代码的能力了。